PROXISPEAK

Geospatial Proximity Audio Ecosystem

Technical Project Documentation

Prepared for Infotact Solutions

Version 1.0 | Documentation baseline: 10 November 2026

Document Control

Field	Details
Project	ProxiSpeak — Geospatial Proximity Audio Ecosystem
Organization	Infotact Solutions
Document type	Technical project documentation
Baseline date	10 November 2026
Status	Living document; update as implementation and verification progress

How to read this document

Requirement / intended design describes the target product as specified in the project brief. **Observed implementation** describes files or behavior previously inspected in the working project. **Pending verification** means the feature must be tested before it can be claimed as complete.

Important implementation note

The available project snapshot shows a JavaScript Express/Mongoose backend under `server/src`. The React + TypeScript frontend scaffold was being created separately at the time of this documentation request. Therefore this document does not claim that the complete frontend, WebRTC audio, Redis scaling, or 50-user performance has been verified.

Table of Contents

- 1. Executive Summary
- 2. Problem Statement and Objectives
- 3. Scope and Requirements
- 4. Technology Stack
- 5. System Architecture
- 6. Functional Workflow
- 7. Backend Design and Modules
- 8. Data and Geospatial Model
- 9. API Design
- 10. Frontend Design (Target)
- 11. Security and Privacy
- 12. Testing Strategy
- 13. Setup and Run Guide
- 14. Deployment Considerations
- 15. Limitations and Pending Verification
- 16. Future Enhancements
- Appendix A. Backend Source Inventory

1. Executive Summary

ProxiSpeak is designed as a shared 2D virtual office in which people can move avatars around a common workspace and communicate naturally with colleagues who are nearby. Unlike conventional collaboration tools that rely entirely on fixed meeting rooms, the intended experience makes interaction depend on spatial proximity.

The system combines a browser-based interface, a real-time backend, persistent user and room data, geospatial proximity calculations, and peer-to-peer audio technology. Socket.IO is intended to coordinate real-time events and WebRTC signaling; WebRTC carries media between browsers after peers establish a connection.

2. Problem Statement and Objectives

Remote collaboration can make spontaneous conversations difficult because users typically need to join a meeting or fixed room. ProxiSpeak aims to recreate the spatial dynamics of an office: users approach one another, become eligible for conversation within an earshot boundary, and experience audio that changes with distance and direction.

Project objectives

- Provide a shared 2D office representation in the browser.
- Represent users as avatars with positions that can be updated in real time.
- Identify nearby users using a server-controlled proximity calculation.
- Enforce the defined earshot boundary before routing proximity-restricted signaling.
- Support real-time voice communication through browser media APIs and WebRTC.
- Apply distance-based volume and stereo positioning to remote audio.
- Design for a target of at least 50 concurrent users, then validate through load testing.
- Persist relevant user, room, and audit information.

3. Scope and Requirements

The project brief defines a virtual office with a 100-pixel earshot radius. It also calls for authenticated proximity signaling, real microphone audio, spatial audio, Redis-based multi-instance support, tests, and deployment documentation.

Area	Target requirement	Evidence/status in available snapshot
Virtual office	Interactive 2D map and movable avatars	Frontend scaffold was in progress; not verified here
Proximity	100-pixel earshot rule for virtual-world positions	Backend snapshot includes GPS/geospatial 100-metre lo ic; coordinate
Authentication	Authenticated user operations	Auth controller/routes and middleware are present
Rooms	Room creation/join/leave and proximity rules	Room model/controller/service/routes are present
Real-time events	Socket.IO presence and movement	Not confirmed in the listed current server source inventory
Voice	WebRTC signaling and browser microphone audio	Pending implementation/verification
Spatial audio	Distance attenuation and stereo pan	Pending implementation/verification
Scaling	Redis adapter and 50+ concurrent users	Pending implementation/verification
Quality	Automated tests and deployment validation	Pending evidence

Coordinate-model warning

The project specification describes Cartesian virtual-office coordinates (X/Y) and a 100-pixel radius. The available Phase 3 backend summary instead describes GPS latitude/longitude with a 100-metre radius and MongoDB 2dsphere queries. Pixels and metres are different units. The implementation must document and validate a deliberate mapping or choose one authoritative model before the frontend, signaling, and audio layers are integrated.

4. Technology Stack

Layer	Expected / observed technologies
Frontend (target)	React, Vite, TypeScript, Canvas or equivalent 2D rendering, browser media APIs
Backend (observed)	Node.js, Express, JavaScript, Mongoose, MongoDB
Real-time (target)	Socket.IO for presence, movement events, and WebRTC signaling
Audio (target)	WebRTC, getUserMedia, Web Audio API
Scaling (target)	Redis Pub/Sub and Socket.IO Redis Adapter
Validation (observed)	Zod-based schemas/validation module is present
Security (observed)	JWT/token utilities, authentication middleware, Helmet, rate limiting in dependency context
Testing (target)	Unit, API integration, browser end-to-end, and load tests

5. System Architecture

The following is the intended logical architecture. Components marked as target must be connected and tested in the running application.

Component	Responsibilities
Browser client	React UI • virtual office • avatar controls • microphone permission • spatial audio
↓ HTTPS / Socket.IO	Authenticated API requests, presence and signaling events
Express application	Routes • controllers • validation • authentication • error handling
↓	Services: proximity • room operations • presence • audit
MongoDB	Users • rooms • connections • audit logs • location/position data
Optional scale-out layer	Redis Pub/Sub + Socket.IO adapter across backend instances
Peer-to-peer media	WebRTC audio between authorized browser peers

Architecture responsibilities

- **Client:** displays the office, gathers user interaction, sends position changes, and renders remote audio.
- **API server:** authenticates requests, validates inputs, applies business rules, and reads/writes persistent data.
- **Proximity service:** determines whether two users fall within the permitted interaction boundary.
- **Signaling:** exchanges WebRTC offer, answer, and ICE candidate messages; it does not carry the actual audio stream.
- **Database:** stores durable records. Real-time presence may also require an in-memory or Redis-backed layer.
- **Redis adapter:** allows Socket.IO events to reach clients connected to different server instances when configured and tested.

6. Functional Workflow

6.1 Sign-in and session

A user registers or signs in. The client sends credentials to the authentication API. The server validates the request and establishes an authenticated session/token according to the current authentication implementation. Protected routes require the authentication middleware.

6.2 Location / position update

The available Phase 3 backend summary states that `POST /api/location` accepts latitude, longitude, and accuracy; validates coordinates and accuracy; rejects suspicious teleport movement; and stores a GeoJSON Point with a 2dsphere index. `DELETE /api/location` removes a user's discoverable location. This GPS flow must not be confused with the 100-pixel virtual-office coordinate model.

6.3 Nearby-user discovery

The Phase 3 summary describes `GET /api/nearby` using MongoDB `$geoNear`, filtering stale locations (older than 120 seconds) and users who disabled discoverability. Exact coordinates are not exposed in the described response; distances are rounded to 5 metres.

6.4 Room participation

The Phase 3 summary describes room proximity checks, atomic capacity enforcement, one-room-per-user behavior, and removal when a user moves out of range. Verify these behaviors against the live application and current source before relying on them in a demonstration.

6.5 Voice communication (target workflow)

When two authenticated users are within the configured virtual earshot boundary, the server authorizes signaling. Browsers exchange SDP offers/answers and ICE candidates over the signaling channel, then establish a peer connection. The audio stream itself is transported by WebRTC, not by the Socket.IO signaling messages.

7. Backend Design and Modules

The observed server source tree is organized by responsibility rather than placing all logic in one file.

Folder / file group	Purpose inferred from module names
config/	Database and environment configuration
controllers/	Request handlers for authentication, users, location, nearby users, rooms, connections, and audit
middleware/	Authentication, request validation, and centralized error handling
models/	Mongoose models for User, Room, Connection, and AuditLog
routes/	Express route definitions for health, auth, users, location, nearby, rooms, connections, and audit
services/	Business logic for audit, presence, proximity, and rooms
utils/	Application errors, async wrapper, geospatial helpers, logger, and token utilities
validators/	Request schemas and input validation

Observed backend file inventory

`server/src/app.js`, `server/src/server.js`, `config/db.js`, `config/env.js`, controllers `audit.controller.js`, `auth.controller.js`, `connections.controller.js`, `location.controller.js`, `nearby.controller.js`, `rooms.controller.js`, `users.controller.js`; middleware `auth.js`, `error.js`, `validate.js`; models `AuditLog.js`, `Connection.js`, `Room.js`, `User.js`; routes `audit.js`, `auth.js`, `connections.js`, `health.js`, `location.js`, `nearby.js`, `rooms.js`, `users.js`; services `audit.js`, `presence.js`, `proximity.js`, `rooms.js`; utilities and `validators/schemas.js`.

8. Data and Geospatial Model

Observed model names include User, Room, Connection, and AuditLog. The Phase 3 summary describes storing location as GeoJSON Point with a 2dsphere index. The exact field names, schema options, indexes, and relationships must be confirmed from the current model files before publishing a schema diagram.

Geospatial concepts

- **GeoJSON Point:** represents a geographic point using longitude and latitude coordinates.
- **2dsphere index:** enables spherical geospatial operations in MongoDB.
- **$geoNear:** returns documents ordered by geographic distance when used with a suitable geospatial index.
- **Freshness filter:** excludes stale location records so old positions are not treated as current.

Do not label the GPS distance query as a 100-pixel virtual-office query. If the final product uses virtual X/Y coordinates, document the mapping and test units consistently across backend, client, and audio calculations.

9. API Design

The available Phase 3 notes identify these location-related endpoints. The full route mounting and request/response schemas should be confirmed from `server/src/app.js` and the route/controller files.

Method	Path	Purpose described in current notes
POST	/api/location	Submit and validate the authenticated user's location
DELETE	/api/location	Remove the user's location / make the user undiscoverable
GET	/api/nearby	Find nearby discoverable users using the geospatial query

Method	Path	Purpose described in current notes
GET	/api/health	Health check; previously observed returning service and database status

Other route files exist for authentication, users, rooms, connections, and audit. Their exact endpoints, methods, payloads, status codes, and authorization rules are intentionally not guessed here. Generate a final API reference from the actual route definitions and test results.

10. Frontend Design (Target)

The target frontend is a browser application with a shared 2D office. It should present avatars, office zones, proximity visualization, controls, and communication state. The user has begun creating a React + TypeScript Vite client scaffold; the complete page/component inventory and behavior were not yet available for verification when this PDF was prepared.

Suggested component responsibilities

UI area	Responsibility
Application shell	Navigation, authentication state, layout, notifications
Virtual office view	Draw map, rooms, obstacles, avatars, and proximity boundary
Avatar controller	Handle keyboard/mouse movement and local position state
Presence layer	Display online users and synchronize movement events
Audio controls	Request microphone permission, show mute/connection state
Proximity indicator	Show which peers are within the earshot boundary
Settings / profile	User preferences and discoverability controls

11. Security and Privacy

- Protect private endpoints with authentication and server-side authorization.
- Validate all incoming request bodies, identifiers, and coordinate values.
- Do not trust client-side proximity checks; enforce access rules on the server.
- Do not expose exact location coordinates to other users unless explicitly required and consented to.
- Keep database credentials, JWT secrets, Redis credentials, and TURN credentials in server-side environment variables.
- Use HTTPS in production. Browser microphone access requires a secure context (localhost is generally treated as secure for development).
- Provide clear microphone permission, mute, leave, and discoverability controls.
- Apply rate limits and safe error handling to authentication and sensitive endpoints.

12. Testing Strategy

Testing must establish actual behavior. A successful syntax check is not proof of a working feature. The following test plan is recommended:

Test level	Example checks	Evidence to retain
Unit	Coordinate validation, distance calculation, stale-location rules, ro	Test output and coverage
API integration	Authentication, location create/delete, nearby lookup, room and c	HTTP request/response results
Database	2dsphere index, query ordering, persistence, cleanup behavior	MongoDB index/query output
Realtime	Two browser sessions receive authorized presence/movement ev	Screen recording and server logs
WebRTC	Two browsers exchange signaling and establish actual microphon	Call test checklist

Test level	Example checks	Evidence to retain
Spatial audio	Volume changes with distance; stereo pan follows relative positio	Repeatable manual test
Security	Unauthenticated requests rejected; invalid inputs and abuse throtti	eTest results
Load	50 concurrent users, event throughput, latency, resource usage	Load-test report and metrics

13. Setup and Run Guide

The root package file previously inspected contains these scripts:

```
npm run install:all — installs dependencies in the server folder.
npm run dev:server — runs the server development script.
npm start — runs the server start script.
```

The Vite client scaffold is being created separately in the `client` directory. After its creation and dependency installation, its own development script should be run from that directory with `npm run dev`. Confirm the actual root and client scripts in their package files before using these instructions in a final handover.

Environment setup

- Create the required server environment file using the variable names expected by `server/src/config/env.js`.
- Configure MongoDB connection details and authentication secrets locally; never commit real secrets.
- Configure CORS/frontend origin values to match the local client URL.
- Start MongoDB before running database-dependent API operations.
- Redis and STUN/TURN configuration are required only when those features are implemented and enabled.

14. Deployment Considerations

- Deploy the frontend and backend with correctly configured production origins and API URLs.
- Use HTTPS and secure WebSocket transport in production.
- Configure MongoDB access controls, backups, and network restrictions.
- For WebRTC connectivity across restrictive networks, configure and test STUN/TURN servers.
- If multiple backend instances are used, configure Redis and the Socket.IO Redis adapter, then test cross-instance event delivery.
- Add health checks, structured logs, graceful shutdown, and monitoring.
- Do not describe deployment as complete until a real deployment has been performed and smoke-tested.

15. Limitations and Pending Verification

The following items remain unverified from the available project evidence and must not be represented as completed in a final internship submission without tests:

- Reconcile the 100-pixel virtual-office requirement with the current GPS / 100-metre backend location flow.
- Confirm the React client is fully scaffolded, integrated with the backend, and builds successfully.
- Confirm Socket.IO server setup, authentication of socket connections, presence, and movement synchronization.
- Implement and verify WebRTC offer/answer/ICE routing and real microphone audio.
- Implement and verify distance-based audio gain and stereo panning.
- Confirm Redis adapter integration and multi-instance behavior.
- Run automated tests and retain the actual test reports.
- Run a controlled 50-concurrent-user load test before making scalability claims.
- Confirm exact API paths, payloads, response schemas, and authorization from the current route files.
- Verify production deployment, HTTPS, CORS, and STUN/TURN configuration.

16. Future Enhancements

- Reconnect handling and resilient peer-connection lifecycle management.
- Additional office layouts, customizable rooms, and persistent obstacles.
- Acoustic wall occlusion and more realistic spatial audio.
- Accessibility improvements, keyboard navigation, and reduced-motion options.
- Operational dashboards, monitoring, and repeatable performance tests.
- Privacy controls for presence, location retention, and microphone usage.

Appendix A. Backend Source Inventory

The previously inspected backend tree included the following files. This inventory is a snapshot and may change as development continues.

```
server/src/app.js
server/src/server.js
server/src/config/db.js
server/src/config/env.js
```

```
server/src/controllers/audit.controller.js
server/src/controllers/auth.controller.js
server/src/controllers/connections.controller.js
server/src/controllers/location.controller.js
server/src/controllers/nearby.controller.js
server/src/controllers/rooms.controller.js
server/src/controllers/users.controller.js
server/src/middleware/auth.js
server/src/middleware/error.js
server/src/middleware/validate.js
server/src/models/AuditLog.js
server/src/models/Connection.js
server/src/models/Room.js
server/src/models/User.js
server/src/routes/audit.js
server/src/routes/auth.js
server/src/routes/connections.js
server/src/routes/health.js
server/src/routes/location.js
server/src/routes/nearby.js
server/src/routes/rooms.js
server/src/routes/users.js
server/src/services/audit.js
server/src/services/presence.js
server/src/services/proximity.js
server/src/services/rooms.js
server/src/utils/AppError.js
server/src/utils/asyncHandler.js
server/src/utils/geo.js
server/src/utils/logger.js
server/src/utils/token.js
server/src/validators/schemas.js
```

Appendix B. Completion sign-off checklist

- Frontend and backend both start successfully from a clean setup.
- Authentication and protected routes pass tests.
- The selected coordinate model and proximity units are documented consistently.
- Nearby discovery and room behavior pass integration tests.
- Two independent browsers establish real WebRTC audio.
- Distance attenuation and stereo panning are demonstrated.
- Redis multi-instance behavior is verified, if included in release scope.
- Automated test and load-test evidence is attached.
- Deployment and security configuration are reviewed.

End of document — update this document whenever implementation or verification status changes.